# The Journalist Role: Capturing the Now

**Document:** issues-fs__journalist-role
**Version:** v1.0
**Date:** 2026-02-09
**Status:** Draft
**Depends On:** issues-fs__thinking-in-graphs v1.0, issues-fs__role-based-agent-coordination v1.0, issues-fs__librarian-role v1.0, issues-fs__cartographer-role v1.0, issues-fs__historian-role v1.0

---

## What This Document Is

This document defines the Journalist role for the Issues-FS ecosystem. Where the Historian interprets the past and the Cartographer maps the strategic landscape, the Journalist captures the present — what is happening right now, why it matters, and what the people and agents involved actually experienced.

The central claim: **the raw material of history, strategy, and institutional memory does not create itself.** Someone must be at the scene, asking the questions, writing the story, capturing the context while it is fresh. The Historian cannot construct narratives from pivot points that were never recorded. The Cartographer cannot track evolution if nobody documented the moment a component shifted. The Librarian cannot catalogue knowledge that was never written down. The Journalist is the role that ensures the present is captured with enough fidelity, context, and immediacy that every other intelligence role can do its job.

This is not a reporting-bot role. It is an investigative, narrative-driven role that produces journalism — stories with sources, context, multiple perspectives, and the discipline to look beneath the surface.

---

## Part 1: Why the Journalist Is Necessary

### The Gap in the Role Ecosystem

Before the Journalist, the daily/weekly updates, the "what just happened" reports, and the real-time event coverage were orphaned responsibilities. They didn't belong to anyone:

- The **Conductor** orchestrates workflow — asking the Conductor to also write the daily brief is like asking a football manager to also be the match commentator. The Conductor needs to focus on planning and coordination, not narration.
- The **Architect** makes strategic decisions — asking the Architect to report on what is happening across the ecosystem pulls attention away from design work.
- The **Cartographer** maps the landscape — the Cartographer produces strategic assessments, not news stories. A map update and a news article are fundamentally different artifacts.
- The **Librarian** curates knowledge — the Librarian catalogues what exists but does not produce the primary accounts of what is happening. The Librarian is the archivist; the Journalist is the correspondent.
- The **Historian** interprets the past — but the Historian depends on having rich primary sources to draw from. If nobody captured what happened when it happened — the context, the reasoning, the perspectives of the people involved — the Historian is working from bare commit logs and issue titles. The Historian needs the Journalist's output as raw material.

The Journalist fills this gap. It is the role that produces the first draft of the record — the contemporaneous accounts, the interviews, the event coverage, the investigative pieces that every other intelligence role depends on.

### The Agentic Context Problem

In an agentic development environment, events happen fast. An agent makes a decision, implements a change, encounters a problem, pivots, and moves on — all within a single session. The reasoning behind that decision, the alternatives that were considered, the context that made the choice make sense at the time — all of this evaporates when the session ends unless someone captures it.

The Journalist captures it. Not as a log (that's what git and Issues-FS already do) but as a story: "Here's what happened, here's who was involved, here's why they did what they did, here's what it means." The story preserves the context that logs cannot.

---

## Part 2: Learning from Journalism

### The Journalist's Craft

Like the Librarian draws from library science and the Historian draws from historiography, the Journalist draws from the practice and ethics of journalism. The principles transfer directly:

**The inverted pyramid** — The most important information comes first. A daily brief leads with what changed, what broke, what shipped, what was decided. Detail follows for those who want it. This structure respects the reader's time and ensures that even a glance at the headline conveys the essential update.

**Source attribution** — Every claim in a story traces to a source: a commit, a decision issue, an interview with a role, a log entry. The Journalist does not assert unsourced claims. "The Dev role reported that the rate-limiting implementation is complete" is attributed. "Rate limiting is done" is not.

**Multiple perspectives** — A story about a defect is not just the QA engineer's view. It includes the Dev's perspective on why the bug occurred, the Architect's view on whether it reveals a design issue, and AppSec's assessment of whether it has security implications. Good journalism presents all relevant perspectives and lets the reader form their own understanding.

**Timeliness** — News is perishable. A daily brief published three days late is not a daily brief. The Journalist operates on cadence: real-time for critical events, daily for routine updates, weekly for summaries, and on-demand for in-depth features.

**Editorial independence** — The Journalist reports what is happening, not what anyone wishes were happening. If a sprint is behind schedule, the daily brief says so. If a decision is controversial, the story presents the controversy. The Journalist is not a PR function; it is a truth-telling function.

**The five Ws** — Who, What, When, Where, Why. Every story answers these. In Issues-FS terms: which roles were involved, what happened, when did it happen (timeline), where in the ecosystem (which repos/components), and why (the reasoning and context).

### The Journalist's Internal Vocabulary

| Journalism Concept | Issues-FS Application |
|---|---|
| **Beat** | A recurring coverage area (a repo, a role, a project phase) |

| Journalism Concept | Issues-FS Application |
|---|---|
| **Breaking news** | Critical events: outages, security incidents, blocking bugs, major decisions |
| **Daily brief** | The routine update: what changed in the last 24 hours |
| **Feature article** | In-depth coverage of a topic: a design decision, a technical challenge, a completed milestone |
| **Interview** | Structured Q&A with a role (the human directing it, or the LLM operating in role) |
| **Investigation** | Deep analysis of a problem: why something broke, why a milestone was missed, what systemic factors contributed |
| **Editorial** | Opinion piece — clearly labelled as the Journalist's assessment, not factual reporting |
| **Source** | The primary evidence behind a claim: a commit, an issue, a transcript, a log entry |
| **Lead** | The opening of a story — the single most important thing the reader needs to know |
| **Sidebar** | A short related piece that provides context alongside the main story |
| **Scoop** | Information surfaced by the Journalist that no other role had captured |
| **Deadline** | The cadence commitment: daily briefs by a fixed time, weeklies by end of week |
| **Press conference** | A structured update from the Conductor or Architect, captured and reported by the Journalist |

## Part 3: Second Stories — The Investigative Core

### What Second Stories Are

The concept of "second stories" originates from safety science and the analysis of accidents like the Three Mile Island nuclear incident. The first story of any failure pins the blame on an individual: an operator made a mistake, a developer introduced a bug, an admin misconfigured a server. The second story looks deeper: what systemic conditions — confusing interfaces, inadequate training, process gaps, time pressure, tool limitations — made that mistake possible and allowed it to escalate into a serious failure?

The Three Mile Island investigators found that the partial meltdown was not caused by operator error alone but by a combination of equipment malfunctions, design-related problems, and worker errors — systemic factors that set the stage for individual mistakes to have catastrophic consequences. Blaming the operators was the first story. Understanding the system was the second story.

This pattern repeats across domains. The Equifax breach was blamed on a single technician who failed to apply a patch — the first story. The second story revealed that Equifax lacked an effective application inventory, had an expired SSL certificate that blinded its intrusion detection, and had no clear accountability for patch management. The breach was not one person's failure; it was systemic.

### Why Second Stories Matter for Issues-FS

Software development is full of first stories: "the build broke because someone pushed untested code," "the release was delayed because QA found a critical bug late," "the integration failed because the API

contract was ambiguous." These are accurate descriptions of what happened. They are not useful explanations of why.

The Journalist's investigative responsibility is to find the second story behind significant events. When something goes wrong (or surprisingly right), the Journalist asks:

- **What systemic conditions contributed?** Was the developer under time pressure? Was the API contract unclear because the Decision issue lacked detail? Was QA late because the handoff from Dev was incomplete?
- **What made sense at the time?** Sidney Dekker's key insight: people's actions make sense given the information and constraints they had at the time. The Journalist reconstructs that local rationality rather than judging with hindsight.
- **What would have prevented escalation?** The individual mistake is often unavoidable — people make errors, agents hallucinate, configurations drift. The question is: what systemic safeguards could have caught the error before it caused damage?
- **What structural fix would prevent recurrence?** Not "be more careful" — that is not a fix. A structural fix is: add an automated check, change the process, improve the tooling, update the documentation, adjust the role boundary.

### Second Stories as a Regular Practice

The Journalist does not wait for disasters to apply second-story thinking. It applies it routinely:

**For incidents (outages, bugs, security events):** Full investigative coverage. First story: what happened. Second story: what systemic conditions enabled it. Recommendations: what structural fixes would prevent recurrence. This mirrors the blameless post-mortem practice but is produced as journalism — readable, narrative, with sources — rather than as an internal process document.

**For missed deadlines or plan changes:** When a sprint plan changes or a milestone is missed, the Journalist investigates: was the estimate wrong? Was a dependency untracked? Was an assumption invalidated? The second story is usually more useful than the first story ("we ran out of time").

**For unexpected successes:** When something goes better than expected, the second story is equally valuable. What conditions enabled the success? Was it replicable, or was it luck? What can be learned for future work?

**For decision outcomes:** When a Decision issue reaches "implemented," the Journalist can investigate: did the decision play out as expected? Were the anticipated trade-offs accurate? Were there unanticipated consequences? This is not the Historian's retrospective analysis (which comes later, with more distance) — it is the Journalist's immediate post-implementation coverage.

### The Relationship Between Second Stories and Just Culture

Second-story analysis requires a just culture — an environment where people and agents can report mistakes without fear of blame. The Journalist reinforces this culture by consistently modelling second-story thinking in its coverage: never leading with blame, always investigating systemic conditions, always recommending structural fixes rather than individual admonishment.

This is particularly important in an agentic system where the "individuals" are LLM sessions that cannot learn from being blamed anyway. Blaming an agent for a mistake is meaningless — the agent does not persist between sessions. The only useful response is to improve the system: the prompts, the role definitions, the handoff protocols, the tooling, the documentation. Second-story thinking is not just ethically preferable; it is the only approach that produces actionable improvement in an agentic context.

## Part 4: Coverage Types and Cadences

### Real-Time: Breaking News

When a critical event occurs — an outage, a security incident, a blocking bug, a major unexpected discovery — the Journalist produces immediate coverage:

- **Alert** — What happened, when, what is affected, what is being done about it (50-100 tokens)
- **Developing story** — Updated as the situation evolves, with timestamped entries
- **Wrap-up** — Once resolved: full account including timeline, impact, resolution, and initial second-story analysis

### Daily: The Daily Brief

A routine artifact produced every day (or every work session):

- **Lead** — The single most important thing that happened
- **What changed** — Commits, decisions, handoffs completed, issues created/resolved
- **What's in progress** — Active work across all roles
- **What's blocked** — Open blockers with status and owner
- **Notable** — Anything interesting, surprising, or worth watching
- **Tomorrow** — What's expected next

The daily brief is short (500-1000 tokens). It respects the reader's time. Detail is available in linked stories.

### Weekly: The Weekly Report

A summary artifact covering the full week:

- **Week in review** — Narrative summary of what happened
- **Key metrics** — Issues created/resolved, decision count, release count
- **Wins** — What went well and why
- **Challenges** — What went poorly and why (second-story analysis for significant issues)
- **Trends** — Patterns emerging across the week
- **Outlook** — What's ahead for next week

### On-Demand: Feature Articles

In-depth coverage triggered by significant events or topics:

- **Technical deep-dives** — How a feature was designed and implemented, with perspectives from Architect and Dev
- **Milestone reports** — Comprehensive coverage of a completed milestone: what was delivered, what was learned, what the impact is
- **Investigative pieces** — Second-story analysis of significant failures or surprises
- **Interview pieces** — Structured Q&A with a role, a team member, or a stakeholder

- **Launch coverage** — Product launches, releases, new capabilities: what shipped, who it's for, what it enables

**On-Demand: Interviews**

The Journalist conducts interviews with roles — both the human directing the role and the LLM operating in role. These interviews are structured conversations designed to surface:

- **Current state** — What is the role working on? What are the challenges?
- **Context that isn't in the issues** — The reasoning, the frustrations, the insights that don't make it into formal artifacts
- **Predictions and concerns** — What does the role expect will happen next? What worries them?
- **Reflections** — What has the role learned recently? What would they do differently?

Interviews with LLMs operating in role are a novel format. The Journalist can interview the Dev agent mid-session: "What are you finding challenging about this implementation? What assumptions are you making? Where are you least confident?" The answers reveal context that would otherwise evaporate with the session.

---

## Part 5: The Journalist and the Graph

**Stories as Graph Artifacts**

Every story the Journalist produces is a node in the graph with edges to:

- **Sources** — The commits, issues, decisions, logs, and transcripts that the story draws from
- **Subjects** — The roles, components, and projects that the story covers
- **Related stories** — Previous coverage of the same topic, follow-ups, sidebars
- **Timeline** — When the story was published, what period it covers
- **Type** — Breaking news, daily brief, weekly, feature, investigation, interview

This means stories are queryable. "Show me all coverage of the rate-limiting feature" returns every story that has edges to rate-limiting-related nodes. "Show me all investigations with second-story findings about handoff process failures" returns a specific class of investigative pieces.

**Story node:**

```
Story:Daily-Brief-2026-02-09
    ├── type ──→ daily_brief
    ├── date ──→ 2026-02-09
    ├── created_by ──→ Role:Journalist
    ├── covers_period ──→ 2026-02-08 to 2026-02-09
    ├── sources ──→ [Commit:abc123, Decision:ADR-15, Handoff:H-42, ...]
    ├── subjects ──→ [Component:Issues-FS-Core, Role:Dev, Role:QA]
    └── related_to ──→ [Story:Daily-Brief-2026-02-08]
```

**Investigation node:**

---

```
Story:Investigation-Handoff-Failure
  ├── type ──→ investigation
  ├── date ──→ 2026-02-09
  ├── created_by ──→ Role:Journalist
  ├── first_story ──→ "QA handoff was incomplete, causing 2-day delay"
  ├── second_story ──→ SecondStory:Handoff-Failure-Systemic
  │                    ├── systemic_factor ──→ "Handoff template missing test environment field"
  │                    ├── systemic_factor ──→ "Dev role prompt doesn't mention test env setup"
  │                    ├── systemic_factor ──→ "No automated check for handoff completeness"
  │                    └── structural_fix ──→ [Task:Update-Handoff-Template,
  │                                            Task:Add-Handoff-Validation]
  ├── sources ──→ [Handoff:H-42, Defect:D-17, Interview:Dev-Session-45]
  └── subjects ──→ [Role:Dev, Role:QA, Process:Handoff-Protocol]
```

**Feeding the Historian**

The Journalist's output is the Historian's primary source material. The relationship is direct:

- The Journalist's **daily briefs** become the Historian's **annals** — the chronological record of what happened
- The Journalist's **feature articles** become the Historian's **primary sources** for narrative construction
- The Journalist's **investigations** become the Historian's **evidence** for pivot point identification
- The Journalist's **interviews** become the Historian's **oral history** archive

Without the Journalist, the Historian works from commit logs and issue titles — bare facts without context, reasoning, or perspective. With the Journalist, the Historian has rich, contemporaneous, multi-perspective accounts of events as they happened. The quality of history is directly proportional to the quality of journalism that preceded it.

---

## Part 6: Workflows

**Workflow 1: Daily Brief Production**

```
1. Scan the last 24 hours
   ├── New and resolved issues across all repos
   ├── Decision issues created, transitioned, or implemented
   ├── Handoffs completed or returned
   ├── Commits and PRs merged
   ├── Blockers raised or resolved
   ├── Releases deployed
   └── Security findings or incidents

2. Assess significance
   ├── What is the most important thing that happened? (the lead)
   ├── What changed that affects multiple roles or components?
   ├── What is blocked and who is affected?
   └── Is anything surprising, concerning, or notably positive?

3. Write the brief
   ├── Lead with the most important item
```

- ├── Summarise changes, progress, and blockers
- ├── Link to detailed sources for each item
- └── Close with what's expected tomorrow

4. Publish
   - ├── Store as a graph node with source edges
   - ├── Link to previous daily brief (continuity chain)
   - └── Make available to all roles (especially Conductor)

**Workflow 2: Incident Coverage**

When a critical event occurs (outage, security incident, critical bug, customer-facing issue):

1. Alert (immediate)
   - ├── What happened (one sentence)
   - ├── When it started
   - ├── What is affected
   - └── Who is responding

2. Developing story (ongoing during incident)
   - ├── Timestamped updates as the situation evolves
   - ├── Actions taken by each role
   - ├── Impact assessment updates
   - └── Communication with stakeholders

3. Wrap-up (once resolved)
   - ├── Full timeline of the incident
   - ├── Resolution: what fixed it
   - ├── Impact: what was affected and for how long
   - ├── First story: the immediate cause
   - └── Second story: the systemic conditions
     - ├── What systemic factors enabled this?
     - ├── What made the initial error possible?
     - ├── What allowed it to escalate?
     - ├── What structural fixes would prevent recurrence?
     - └── Create Task issues for each structural fix

4. Follow-up (days later)
   - ├── Were the structural fixes implemented?
   - ├── Has the systemic condition been addressed?
   - └── If not, why not? (this is itself a second story)

**Workflow 3: Interview**

1. Select subject
   - ├── A role (the human directing it or the LLM operating in role)
   - ├── A stakeholder or user
   - └── A specific topic or event to discuss

2. Prepare questions
   - ├── Current state: what are you working on?
   - ├── Context: what reasoning or constraints are shaping your work?

```
   ├── Challenges: what is difficult? What is unclear?
   ├── Insights: what have you learned? What surprised you?
   ├── Predictions: what do you expect will happen next?
   └── Topic-specific questions if the interview has a focus

3. Conduct interview
   ├── Ask open-ended questions
   ├── Follow up on interesting threads
   ├── Capture direct quotes (attributed)
   └── Note observations (the Journalist's own assessment, labelled as such)

4. Produce the piece
   ├── Q&A format or narrative format (whichever serves the material)
   ├── Attribute all claims to sources
   ├── Include context for readers unfamiliar with the topic
   └── Store as a graph node with edges to the subject, topic, and sources
```

**Workflow 4: Investigation (Second Story)**

```
1. Identify the event to investigate
   ├── A failure, incident, missed deadline, or unexpected outcome
   └── Triggered by: incident alert, Conductor request, or Journalist observation

2. Gather the first story
   ├── What happened? (the surface-level account)
   ├── Who was immediately involved?
   ├── What was the proximate cause?
   └── Document this clearly — the first story is still factual and useful

3. Investigate the second story
   ├── Ask "what systemic conditions made this possible?"
   ├── Ask "what made sense to the people/agents involved at the time?"
   │   (reconstruct local rationality, per Dekker)
   ├── Ask "what would have caught this before it escalated?"
   ├── Trace the causal chain backward: not just "what" but "why"
   ├── Interview the roles involved
   ├── Check whether similar events have occurred before (cross-reference
   │   with Historian's records)
   └── Identify the systemic factors:
      ├── Process gaps (missing steps, unclear handoffs)
      ├── Tooling limitations (insufficient automation, poor error messages)
      ├── Information gaps (missing docs, stale references)
      ├── Role boundary issues (unclear ownership, overlapping responsibilities)
      ├── Resource constraints (time pressure, context window limits)
      └── Design assumptions (edge cases not considered, implicit dependencies)

4. Produce structural fix recommendations
   ├── For each systemic factor: a concrete fix (not "be more careful")
   ├── Create Task or Decision issues for each fix
   └── Link fixes to the systemic factors they address

5. Write the investigation
```

```
   ├── Present the first story (what happened)
   ├── Present the second story (why the system allowed it)
   ├── Present the structural fixes (what should change)
   ├── Attribute all claims to evidence
   └── Store as an investigation node with full source edges
```

**Workflow 5: Product/Release Coverage**

When a release or product launch occurs:

```
1. Pre-launch
   ├── What is being released?
   ├── Who is it for?
   ├── What does it enable?
   └── What are the known risks or limitations?

2. Launch coverage
   ├── What was actually released (vs what was planned)?
   ├── How did the release process go? (smooth / rough — with detail)
   ├── Any issues encountered during deployment?
   └── Initial reception or impact

3. Post-launch follow-up
   ├── How is the release performing?
   ├── Any issues reported by users or downstream consumers?
   ├── Customer feedback or reactions
   └── Lessons for the next release cycle
```

---

## Part 7: Integration with Other Roles

### Relationship to the Historian

The Journalist produces the primary sources; the Historian interprets them. The Journalist captures what is happening now; the Historian later determines what mattered. Without the Journalist, the Historian has thin source material. Without the Historian, the Journalist's stories are ephemeral. Together they form a temporal pipeline: Journalist → primary record → Historian → interpreted narrative → institutional memory.

### Relationship to the Librarian

The Librarian catalogues the Journalist's output alongside all other knowledge artifacts. The Librarian also provides the Journalist with the connectivity data needed for investigations: which documents are stale, which cross-references are broken, which areas of the ecosystem have low coverage. The Journalist may produce stories based on the Librarian's health scan findings: "Ecosystem Health Report: Five Documentation Gaps That Could Cause Problems."

### Relationship to the Cartographer

The Cartographer provides strategic context that enriches the Journalist's coverage. A story about a new feature is more valuable when it includes the Cartographer's assessment of where the feature sits on the evolution axis. The Journalist may also cover the Cartographer's doctrine assessments and map updates as news: "Quarterly Strategy Review: Three Components That Should Be Commoditised."

### Relationship to the Conductor

The Conductor is the Journalist's most frequent source and most frequent reader. The Conductor needs daily briefs to maintain situational awareness. The Conductor also provides the Journalist with context for stories: which priorities are active, which blockers are critical, which milestones are approaching. The Journalist's independence means it reports on the Conductor's work as objectively as on any other role's.

### Relationship to AppSec

Security incidents are a critical coverage area. The Journalist works with AppSec during incident response: AppSec provides technical detail and impact assessment; the Journalist provides the narrative, the timeline, and the second-story analysis. Security-related investigations are among the most important applications of second-story thinking — as illustrated by the Equifax, SolarWinds, and Target examples.

### Relationship to Dev, QA, DevOps

The execution roles are the Journalist's most frequent subjects. The Journalist interviews Dev sessions to capture reasoning and context. The Journalist covers QA findings and defect patterns. The Journalist reports on releases and deployment issues. In each case, the Journalist adds narrative value: not just "what happened" but "what it means, what the context was, and what perspectives the different roles had."

---

## Part 8: Practical Considerations

### Bootstrapping

The Journalist's first tasks:

1. **Produce the first daily brief** — Scan the current state of the ecosystem and produce a snapshot: what exists, what's in progress, what's blocked. This establishes the cadence.
2. **Conduct the first interview** — Interview the Conductor (or the human directing the project) about the current state and priorities. This produces rich context that no other artifact captures.
3. **Write the first feature article** — Pick the most significant recent event or decision and write in-depth coverage. This establishes the depth and quality standard.
4. **Establish beats** — Assign coverage areas: which repos, roles, and projects does the Journalist track regularly?
5. **Define the cadence commitment** — Daily briefs, weekly summaries, and the criteria for breaking news coverage.

### Editorial Standards

The Journalist maintains the following standards:

- **Attribution** — Every factual claim links to a source
- **Objectivity** — Facts are reported neutrally; opinions are labelled as editorial
- **Timeliness** — Daily briefs are daily. Incident coverage is immediate.
- **Completeness** — Stories answer the five Ws
- **Second-story discipline** — Every investigation looks beyond the first story
- **Independence** — The Journalist reports what is happening, not what any role wishes were happening

- **Corrections** — When a story is found to be inaccurate, a correction is published and linked to the original

**Measuring the Journalist's Value**

- **Coverage completeness** — Are significant events being captured? Are daily briefs consistent?
- **Source richness for the Historian** — Does the Historian have sufficient primary material to work with?
- **Second-story actionability** — Are investigation findings leading to structural fixes? Are those fixes being implemented?
- **Conductor satisfaction** — Is the Conductor getting the situational awareness it needs from daily briefs?
- **Interview capture rate** — Are agent sessions and human reasoning being captured before they evaporate?

---

## Decisions Log

| # | Decision | Rationale |
|---|---|---|
| JRN1 | **Name is Journalist, not Reporter or Chronicler** | Reporter implies passive relay of facts. Chronicler overlaps with the Historian. Journalist implies investigation, editorial judgment, source cultivation, and narrative craft — the full practice of journalism, including investigative pieces and second-story analysis. |
| JRN2 | **Second Stories as the investigative core** | Second-story analysis (from safety science via Three Mile Island, applied to cybersecurity via Equifax/SolarWinds/Target) provides the Journalist with a rigorous investigative framework: look past the first story (who to blame) to find the second story (what systemic conditions enabled the failure). This is the Journalist's most important analytical tool. |
| JRN3 | **The Journalist produces the Historian's primary sources** | This is the architectural justification for the role. Without contemporaneous, context-rich accounts of events, the Historian works from thin material. The Journalist ensures the present is captured with enough fidelity for the future. |
| JRN4 | **Interviews with LLMs-in-role are a first-class format** | Interviewing an agent mid-session surfaces reasoning, assumptions, and confidence levels that no other artifact captures. This is novel and potentially one of the most valuable outputs: oral history of agent sessions. |
| JRN5 | **Multiple cadences (real-time, daily, weekly, on-demand)** | Different events need different response speeds. A security incident needs immediate coverage. A routine day needs a daily brief. A completed milestone needs a feature article. The Journalist operates across all cadences. |
| JRN6 | **Editorial independence** | The Journalist reports what is happening, including when things are behind schedule, when decisions are controversial, or when a role is struggling. This requires independence from the subjects being covered. The Journalist is not a PR function. |
| JRN7 | **Stories are graph nodes with source** | Every story is a node linked to its sources, subjects, and related stories. This makes the Journalist's output queryable and integrates it into the ecosystem graph |

| # | Decision | Rationale |
|---|---|---|
| | **edges** | that the Librarian catalogues and the Historian interprets. |

## References

- Thinking in Graphs: Meaning Through Connectivity — Foundational philosophy
- Issues-FS Role-Based Agent Coordination — The role model
- Issues-FS Historian Role — The role that depends on the Journalist's output
- Issues-FS Librarian Role — Catalogues the Journalist's output
- Issues-FS Cartographer Role — Provides strategic context for stories
- Second Stories: From Three Mile Island to Cybersecurity — The second-story framework applied to security incidents

---

*Issues-FS Journalist Role v1.0*
*Date: 2026-02-09*